

MANUAL TECNICO

PRACTICA: DESARROLLOWEB 1era Fase

202400818
Jeremy Isai
Hernández Girón

Introducción

Este proyecto materializa el backend de un Ecosistema de Gestión Académica, diseñado bajo una arquitectura de micro servicios orientada a la eficiencia y la seguridad. El núcleo del sistema se basa en un servidor Node.js con el framework Express, que actúa como el motor de reglas de negocio y puente de comunicación con una base de datos relacional en MySQL. Más allá de un simple CRUD, la solución integra un esquema de autenticación stateless mediante JSON Web Tokens (JWT), garantizando que el flujo de datos entre el cliente y el servidor sea privado y verificado en cada petición. El objetivo principal es ofrecer una plataforma escalable donde la gestión de cursos, catedráticos y la interacción estudiantil ocurra de forma fluida y protegida.

Descripción General del Sistema

Este proyecto es, en palabras simples, el cerebro de una aplicación académica. Su función es recibir peticiones de un usuario (como "quiero ver mis cursos" o "quiero publicar un comentario") y dar una respuesta correcta usando los datos guardados.

Registro y Control: Los usuarios pueden crear su cuenta y entrar al sistema de forma segura, ya que sus contraseñas no se guardan como texto normal, sino "disfrazadas" con un código especial para que nadie más las vea.

Muro de Opiniones: Existe un espacio donde se pueden subir publicaciones y comentarios sobre las clases. Esto ayuda a crear una guía real basada en la experiencia de otros alumnos.

Historial de Cursos: El sistema lleva el control de qué clases ya pasaste. Lo mejor es que el programa es "inteligente": si intentas meter el mismo curso dos veces, te avisa que ya existe, y solito va sumando cuántos créditos llevas acumulados.

Tecnologías utilizadas

El sistema fue desarrollado utilizando tecnologías modernas orientadas a servicios web.

1. Node.js (entorno de ejecución)
2. Express (framework para API REST)
3. MySQL (gestión de base de datos)
4. JWT (autenticación segura)
5. bcrypt (protección de contraseñas)
6. Postman (pruebas del API)

Base de datos

Se creó la base de datos 'informe4_web' con las siguientes tablas:

- 1 usuarios: almacena información de usuarios
- 2 cursos: catálogo de cursos
- 3 catedraticos: catálogo de docentes
- 4 publicaciones: opiniones sobre cursos/catedráticos
- 5 comentarios: interacción entre usuarios
- 6 cursos_aprobados: relación usuario-curso

Las tablas están relacionadas mediante claves foráneas para mantener integridad referencial.

ENDPOINTS

1. POST /api/auth/registro - Registro de usuario
2. POST /api/auth/login - Inicio de sesión
3. GET /api/cursos - Obtener cursos
4. POST /api/publicaciones - Crear publicación
5. GET /api/publicaciones - Listar publicaciones
6. GET /api/publicaciones?nombre_curso=... - Filtrar por curso
7. GET /api/publicaciones/:id/comentarios - Ver comentarios
8. POST /api/publicaciones/:id/comentarios - Agregar comentario
9. POST /api/cursos-aprobados - Registrar curso aprobado
10. GET /api/usuarios/:id/cursos-aprobados - Ver cursos y créditos

AUTENTICACIÓN

POST

http://localhost:3001/api/auth/registro

Registra un nuevo usuario.

POST

http://localhost:3001/api/auth/login

Inicia sesión y retorna un token JWT.

POST

http://localhost:3001/api/auth/recuperar

Permite actualizar la contraseña usando registro y correo.

USUARIOS

GET

<http://localhost:3001/api/usuarios/:registro>

Obtiene la información de un usuario por su registro.

PUT

<http://localhost:3001/api/usuarios/:id>

Permite actualizar los datos del usuario.

CURSOS Y CATEDRÁTICOS

GET

<http://localhost:3001/api/cursos>

Lista todos los cursos disponibles.

GET

<http://localhost:3001/api/catedraticos>

Lista todos los catedráticos.

PUBLICACIONES

GET

<http://localhost:3001/api/publicaciones>

Obtiene todas las publicaciones.

POST

<http://localhost:3001/api/publicaciones>

Crea una nueva publicación.

GET

(filtro por curso)

http://localhost:3001/api/publicaciones?nombre_curso=Programación

GET

(filtro por catedrático)

http://localhost:3001/api/publicaciones?nombre_catedratico=Roberto

COMENTARIOS

GET

`http://localhost:3001/api/publicaciones/:id/comentarios`

Obtiene comentarios de una publicación.

POST

`http://localhost:3001/api/publicaciones/:id/comentarios`

Agrega un comentario.

CURSOS APROBADOS

POST

`http://localhost:3001/api/cursos-aprobados`

Agrega un curso aprobado al usuario.

GET

`http://localhost:3001/api/usuarios/:id/cursos-aprobados`

Obtiene cursos aprobados y total de créditos.

Lógica Implementada

Para que el código no fuera un desorden, se separó por tareas. Una parte recibe los mensajes, otra revisa si el usuario tiene permiso de estar ahí y otra se pelea con la base de datos para traer la información. El Guardián (Middleware): Antes de dejarte entrar a las partes importantes, hay una función que revisa tu "token" de seguridad. Si el token es viejo o no existe, el sistema te rebota. Búsquedas Fáciles: Si quieres buscar un curso específico, no tienes que escribir el nombre exacto. La lógica que programamos permite que, con solo poner una parte del nombre, el sistema encuentre todas las coincidencias. Cero Duplicados: En la parte de cursos aprobados, pusimos reglas para que la base de datos se mantenga limpia. Esto evita que los datos se mezclen o que el conteo de créditos salga mal.

Conclusión

Al terminar este backend, logramos tener una plataforma que funciona bien, es segura y, sobre todo, es útil. Se usaron herramientas que son estándar en la industria hoy en día, lo que garantiza que el sistema no se va a quedar trabado si muchos estudiantes lo usan al mismo tiempo. Lo más importante es que ya tenemos la estructura base. Todo lo que es la seguridad con JWT y la organización de las tablas en MySQL quedó listo para que, en el siguiente paso, solo tengamos que diseñar la parte visual y conectarla. Es un proyecto sólido que demuestra cómo la tecnología puede ayudar a organizar mejor nuestra información en la universidad.